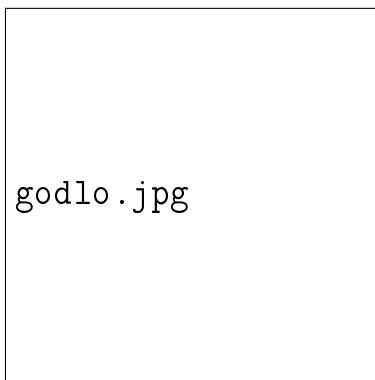


Collegium Witelona Uczelnia Państwowa w Legnicy
Wydział Nauk Technicznych i Ekonomicznych
Kierunek: Informatyka



Projekt do kursu Podstawy Symulacji Komputerowej”

Autor

Gabriela Grabarska, nr indeksu: 43840

Prowadzący przedmiot
dr inż. Ryszard Rębowski

Legnica, 2026

Spis treści

1 Wprowadzenie

Celem projektu jest opracowanie trzech tematów z list zadań z kursu Podstawy Symulacji Komputerowej. Projekt obejmuje część merytoryczną, implementację algorytmów oraz prezentację wyników końcowych w aplikacji komputerowej.

W pracy przedstawiono podstawowe zagadnienia związane z symulacją komputerową, generowaniem liczb pseudolosowych oraz wykorzystaniem tych liczb do otrzymywania rozkładów prawdopodobieństwa. Projekt został przygotowany tak, aby połączyć opis teoretyczny z praktycznym działaniem algorytmów. Dla każdego wybranego zadania opisano podstawy merytoryczne, sposób implementacji oraz otrzymane wyniki.

Pierwsza część projektu dotyczy analizy rozkładu łącznego dwuwymiarowego wektora losowego. W tej części obliczane są rozkłady brzegowe, wartości oczekiwane, kowariancja, współczynnik korelacji oraz sprawdzana jest niezależność stochastyczna i zależność liniowa zmien-nych losowych.

Druga część projektu dotyczy generowania liczb pseudolosowych. W aplikacji zaprezentowano dwa sposoby generowania takich liczb: liniowy generator kongruencyjny Lehmera oraz metodę środkowych kwadratów von Neumanna. Generator Lehmera został następnie wykorzystany jako źródło liczb jednostajnych w zadaniach symulacyjnych.

Trzecia część projektu obejmuje generowanie rozkładów prawdopodobieństwa na podstawie liczb pseudolosowych. Dla rozkładu dyskretnego zaimplementowano generowanie rozkładu Poissona, natomiast dla rozkładu ciągłego zastosowano metodę odwracania dystrybuanty.

Do realizacji projektu przygotowano aplikację webową. Część obliczeniowa została wydzielona do backendu w języku Python, natomiast część prezentacyjna została wykonana w React. Dzięki temu interfejs użytkownika służy do wprowadzania parametrów i prezentacji wyników, a właściwe obliczenia wykonywane są po stronie backendu.

Wybrane tematy:

1. Lista 0, zadanie 9 – analiza rozkładu łącznego wektora losowego.
2. Lista 1, zadanie 8 – generowanie rozkładu Poissona.
3. Lista 2, zadanie 2 – generowanie rozkładu ciągłego metodą odwracania dystrybuanty.

2 Generator LCG Lehmera

W zadaniach z Listy 1 i Listy 2 wykorzystano generator kongruencyjny Lehmera jako źródło liczb pseudolosowych o rozkładzie jednostajnym na przedziale $[0, 1)$. Wygenerowane wartości U_j stanowią dane wejściowe dla algorytmu generowania rozkładu Poissona oraz dla metody odwracania dystrybuanty.

Generator Lehmera tworzy ciąg liczb pseudolosowych według zależności:

$$X_j = aX_{j-1} \pmod{m}, \quad (1)$$

gdzie X_0 oznacza ziarno generatora, a oznacza mnożnik, m oznacza moduł, a X_j oznacza kolejny stan generatora.

Aby uzyskać liczby z przedziału $[0, 1)$, każdy wygenerowany stan jest normalizowany:

$$U_j = \frac{X_j}{m}. \quad (2)$$

W aplikacji wartość modułu wyznaczana jest na podstawie zadanego okresu k :

$$L = \text{round}(\log_2(k) + 2), \quad (3)$$

$$m = 2^L. \quad (4)$$

Dla modułu postaci $m = 2^L$ warunki poprawnego doboru parametrów obejmują:

- $L > 4$,
- $k \geq 100$,
- $a \bmod 8 \in \{3, 5\}$,
- X_0 jest liczbą nieparzystą.

W aplikacji jako domyślne parametry przyjęto:

$$k = 536870912, \quad a = 1103515245, \quad X_0 = 12345, \quad n = 1200. \quad (5)$$

Dla tych wartości otrzymujemy:

$$L = 31, \quad m = 2^{31}, \quad k = 2^{29} = 536870912. \quad (6)$$

Mnożnik spełnia warunek $a \bmod 8 = 5$, a ziarno $X_0 = 12345$ jest nieparzyste.

2.1 Obserwacje i wnioski dotyczące doboru parametrów

Początkowo w aplikacji zastosowano parametry:

$$k = 100, \quad a = 101, \quad X_0 = 3, \quad n = 900. \quad (7)$$

Dla tych wartości generator spełniał podstawowe warunki z zadania, jednak na wykresach 2D i 3D widoczne były wyraźne linie oraz regularne układy punktów.

Problem wynikał głównie z małego okresu generatora. Dla $k = 100$ otrzymywano:

$$L = 9, \quad m = 2^9 = 512, \quad \text{okres} = 2^7 = 128. \quad (8)$$

Jednocześnie liczba generowanych punktów wynosiła $n = 900$, czyli była znacznie większa niż okres generatora. W konsekwencji ciąg zaczynał się wielokrotnie powtarzać, a ta powtarzalność była widoczna na wykresach jako linie i regularne struktury.

Po zgłębieniu tematu generatorów liniowych kongruencyjnych zauważono, że jakość generatora silnie zależy od doboru parametrów a , m oraz X_0 . Zbyt mały moduł lub niekorzystnie dobrany mnożnik może prowadzić do wyraźnej struktury kratowej punktów.

W opracowaniu Karla Entachera *A collection of selected pseudorandom number generators with linear structures* przedstawiono klasyczny generator używany w ANSI C:

$$LCG(2^{31}, 1103515245, 12345, 12345). \quad (9)$$

Na tej podstawie przyjęto większy moduł związany z wartością $m = 2^{31}$ oraz znany mnożnik $a = 1103515245$.

W projekcie nie zastosowano dokładnie generatora ANSI C, ponieważ wzór z zadania dotyczy generatora multiplikatywnego Lehmera:

$$X_j = aX_{j-1} \pmod{m}, \quad (10)$$

a nie generatora mieszanego:

$$X_j = aX_{j-1} + b \pmod{m}. \quad (11)$$

Z przytoczonego przykładu wykorzystano więc ideę dużego modułu 2^{31} oraz znanego mnożnika 1103515245, natomiast składnik addytywny nie został użyty.

Aby w konstrukcji stosowanej w aplikacji uzyskać $m = 2^{31}$, dobrano:

$$L = 31, \quad k = 2^{L-2} = 2^{29} = 536870912. \quad (12)$$

Nowa liczba generowanych punktów jest bardzo mała w porównaniu z okresem generatora:

$$1200 \ll 536870912. \quad (13)$$

Dzięki temu ciąg nie powtarza się szybko, a punkty na wykresach 2D i 3D są znacznie lepiej rozproszone. Należy jednak pamiętać, że każdy generator LCG posiada pewną strukturę kratową, dlatego całkowite usunięcie zależności liniowych nie jest możliwe. Można jedynie dobrać parametry tak, aby przy analizowanej liczbie próbek struktura ta nie była widoczna na wykresach.

2.2 Fragment implementacji

```
@dataclass
class LehmerGenerator:
    seed: int
    modulus: int
    multiplier: int

    def __post_init__(self) -> None:
        self.current_x = int(self.seed or 1)
        self.modulus = int(self.modulus)
        self.multiplier = int(self.multiplier)

    def next_int(self) -> int:
        self.current_x = (self.multiplier * self.current_x) % self.modulus
        return self.current_x

    def next_float(self) -> float:
        return self.next_int() / self.modulus
```

3 Drugi sposób generowania liczb pseudolosowych: metoda von Neumanna

Oprócz generatora LCG Lehmera w aplikacji przedstawiono również algorytm kwadratowy von Neumanna, nazywany metodą środkowych kwadratów. Jest to jedna z historycznych metod generowania liczb pseudolosowych. Jej działanie polega na tym, że kolejną liczbę ciągu otrzymuje się przez podniesienie poprzedniej wartości do kwadratu, a następnie wybranie środkowych cyfr otrzymanego wyniku.

Algorytm przyjmuje wartość początkową X_0 oraz liczbę cyfr m , która określa długość generowanych liczb. Dla każdego kroku:

1. pobierana jest poprzednia wartość X_{n-1} ,
2. obliczany jest kwadrat:

$$Y_n = X_{n-1}^2, \quad (14)$$

3. wynik jest uzupełniany zerami z lewej strony tak, aby miał $2m$ cyfr,
4. ze środka zapisu liczby wybieranych jest m cyfr,
5. otrzymana liczba staje się nową wartością X_n .

Schemat można zapisać następująco:

$$X_{n-1} \rightarrow X_{n-1}^2 \rightarrow \text{środkowe } m \text{ cyfr} \rightarrow X_n. \quad (15)$$

Przykładowo dla $X_0 = 12$ oraz $m = 2$:

$$12^2 = 144 \rightarrow 0144 \rightarrow 14, \quad (16)$$

$$14^2 = 196 \rightarrow 0196 \rightarrow 19, \quad (17)$$

$$19^2 = 361 \rightarrow 0361 \rightarrow 36. \quad (18)$$

Zaletą tej metody jest prostota i łatwość wizualnego pokazania kolejnych kroków obliczeń. W aplikacji każdy krok prezentowany jest osobno: wartość poprzednia, kwadrat, uzupełniony zapis, wyróżnione cyfry środkowe oraz nowa wartość ciągu. Należy jednak zauważyć, że generator von Neumanna ma istotne ograniczenia praktyczne. Dla niektórych wartości początkowych ciąg może szybko wejść w krótki cykl albo osiągnąć wartość 0, po której dalsze generowanie daje stale 0.

3.1 Fragment implementacji

```
def calculate_von_neumann(seed: int, digits: int, count: int) -> list[dict]:
    current_x = int(seed)
    results = []

    for index in range(count):
        square = current_x**2
        square_text = str(square).zfill(2 * digits)
        start = digits // 2
        middle = square_text[start : start + digits]
        value = int(middle)

        results.append({
            "prev": current_x,
            "square": square,
            "full": square_text,
            "middle": middle,
            "value": value,
        })

        current_x = value

        if current_x == 0:
            break

    return results
```

4 Lista 0, zadanie 9

4.1 Treść zadania

Dana jest macierz P reprezentująca rozkład łączny wektora losowego (X, Y) , gdzie:

$$P = \begin{bmatrix} 0,1 & 0,2 & 0,3 \\ 0,1 & 0,1 & 0,2 \end{bmatrix}, \quad (19)$$

oraz:

$$X(\Omega) = \{1, 2\}, \quad Y(\Omega) = \{3, 2, 1\}. \quad (20)$$

Należy obliczyć:

- $\text{cov}(X, Y)$,
- $\rho(X, Y)$,
- sprawdzić, czy zmienne losowe X i Y są stochastycznie niezależne,
- sprawdzić, czy zmienne losowe X i Y są liniowo zależne.

4.2 Podstawy merytoryczne i obliczenia

Rozkład łączny wektora losowego opisuje prawdopodobieństwa jednoczesnego przyjmowania wartości przez dwie zmienne losowe:

$$p_{ij} = P(X = x_i, Y = y_j). \quad (21)$$

Rozkłady brzegowe wyznacza się przez sumowanie odpowiednich wierszy i kolumn:

$$P(X = x_i) = \sum_j p_{ij}, \quad (22)$$

$$P(Y = y_j) = \sum_i p_{ij}. \quad (23)$$

Dla danych z zadania:

$$P(X = 1) = 0,1 + 0,2 + 0,3 = 0,6, \quad (24)$$

$$P(X = 2) = 0,1 + 0,1 + 0,2 = 0,4, \quad (25)$$

$$P(Y = 3) = 0,1 + 0,1 = 0,2, \quad (26)$$

$$P(Y = 2) = 0,2 + 0,1 = 0,3, \quad (27)$$

$$P(Y = 1) = 0,3 + 0,2 = 0,5. \quad (28)$$

Wartości oczekiwane:

$$E(X) = 1 \cdot 0,6 + 2 \cdot 0,4 = 1,4, \quad (29)$$

$$E(Y) = 3 \cdot 0,2 + 2 \cdot 0,3 + 1 \cdot 0,5 = 1,7. \quad (30)$$

Dalej:

$$E(XY) = \sum_i \sum_j x_i y_j p_{ij} \quad (31)$$

$$= 1 \cdot 3 \cdot 0,1 + 1 \cdot 2 \cdot 0,2 + 1 \cdot 1 \cdot 0,3 \quad (32)$$

$$+ 2 \cdot 3 \cdot 0,1 + 2 \cdot 2 \cdot 0,1 + 2 \cdot 1 \cdot 0,2 \quad (33)$$

$$= 2,4. \quad (34)$$

Kowariancja:

$$\text{cov}(X, Y) = E(XY) - E(X)E(Y) \quad (35)$$

$$= 2,4 - 1,4 \cdot 1,7 \quad (36)$$

$$= 0,02. \quad (37)$$

Wariancje:

$$E(X^2) = 1^2 \cdot 0,6 + 2^2 \cdot 0,4 = 2,2, \quad (38)$$

$$\text{Var}(X) = 2,2 - 1,4^2 = 0,24, \quad (39)$$

$$E(Y^2) = 3^2 \cdot 0,2 + 2^2 \cdot 0,3 + 1^2 \cdot 0,5 = 3,5, \quad (40)$$

$$\text{Var}(Y) = 3,5 - 1,7^2 = 0,61. \quad (41)$$

Współczynnik korelacji:

$$\rho(X, Y) = \frac{\text{cov}(X, Y)}{\sqrt{\text{Var}(X) \text{Var}(Y)}} \quad (42)$$

$$= \frac{0,02}{\sqrt{0,24 \cdot 0,61}} \quad (43)$$

$$\approx 0,0522. \quad (44)$$

Warunek niezależności stochastycznej:

$$P(X = x_i, Y = y_j) = P(X = x_i)P(Y = y_j). \quad (45)$$

Dla pary $(X = 1, Y = 3)$:

$$P(X = 1, Y = 3) = 0,1, \quad (46)$$

$$P(X = 1)P(Y = 3) = 0,6 \cdot 0,2 = 0,12. \quad (47)$$

Ponieważ $0,1 \neq 0,12$, zmienne losowe X i Y nie są stochastycznie niezależne.

Zmienne nie są również liniowo zależne, ponieważ dla $X = 1$ zmienna Y może przyjmować kilka różnych wartości z dodatnim prawdopodobieństwem.

4.3 Fragment implementacji

```
expected_xy = 0.0
for row_index, x in enumerate(x_values):
    for column_index, y in enumerate(y_values):
        expected_xy += x * y * probabilities[row_index][column_index]

covariance = expected_xy - expected_x * expected_y
rho = covariance / sqrt(variance_x * variance_y)
```

5 Lista 1, zadanie 8

5.1 Treść zadania

Zaprogramować algorytm generowania rozkładu Poissona. Przyjąć:

$$\lambda = 2,0. \quad (48)$$

W projekcie zadanie wykonano przez wygenerowanie ciągu liczb jednostajnych U_i za pomocą generatora LCG Lehmera, a następnie przekształcenie ich do wartości zmiennej losowej o rozkładzie Poissona.

5.2 Podstawy merytoryczne

Rozkład Poissona jest rozkładem dyskretnym opisującym liczbę zdarzeń zachodzących w ustalonym przedziale czasu lub przestrzeni. Zmienna losowa X ma rozkład Poissona z parametrem λ , jeżeli:

$$P(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, 2, \dots \quad (49)$$

Dla wartości z zadania:

$$P(X = k) = e^{-2} \frac{2^k}{k!}, \quad k = 0, 1, 2, \dots \quad (50)$$

Dla rozkładu Poissona:

$$E(X) = \lambda, \quad \text{Var}(X) = \lambda. \quad (51)$$

Zatem dla $\lambda = 2$:

$$E(X) = 2, \quad \text{Var}(X) = 2. \quad (52)$$

Do wygenerowania rozkładu wykorzystano metodę odwrotnej dystrybuanty dla rozkładu dyskretnego. Najpierw generowana jest liczba jednostajna U z przedziału $[0, 1)$, a następnie szukana jest najmniejsza liczba całkowita k , dla której:

$$F(k) \geq U, \quad (53)$$

gdzie:

$$F(k) = P(X \leq k) = \sum_{i=0}^k e^{-\lambda} \frac{\lambda^i}{i!}. \quad (54)$$

Dla $\lambda = 2$:

$$F(k) = \sum_{i=0}^k e^{-2} \frac{2^i}{i!}. \quad (55)$$

5.3 Algorytm

1. Wygenerować wartość U algorytmem LCG Lehmera.
2. Ustawić $p_0 = e^{-\lambda}$ oraz $F = p_0$.
3. Jeśli $U \leq F$, zwrócić wartość 0.
4. W przeciwnym razie obliczać kolejne prawdopodobieństwa rekurencyjnie:

$$p_k = p_{k-1} \frac{\lambda}{k}. \quad (56)$$

5. Sumować wartości do dystrybuanty F .
6. Zwrócić pierwsze k , dla którego $U \leq F$.

Dla $\lambda = 2$ rekurencja ma postać:

$$p_0 = e^{-2}, \quad p_k = p_{k-1} \frac{2}{k}. \quad (57)$$

5.4 Fragment implementacji

```
def _poisson_from_uniform(uniform_value: float, lambda_value: float) -> tuple[int, float]:
    probability = exp(-lambda_value)
    cumulative = probability
    value = 0

    while uniform_value > cumulative:
        value += 1
        probability *= lambda_value / value
        cumulative += probability

    return value, cumulative
```

5.5 Wyniki końcowe

Aplikacja prezentuje wygenerowane wartości rozkładu Poissona, histogram częstości, średnią z próby, wariancję z próby oraz porównanie z wartościami teoretycznymi $E(X) = \lambda$ i $\text{Var}(X) = \lambda$. Dla domyślnych parametrów $\lambda = 2,0$ oczekuje się, że przy większej liczbie próbek średnia i wariancja z próby będą zbliżać się do wartości 2.

6 Lista 2, zadanie 2

6.1 Treść zadania

Zaimplementować metodologię metody odwracania dystrybuanty, wykorzystując wyniki Przykładu 1 z materiałów wykładowych.

W projekcie jako przykład rozkładu ciągłego zastosowano rozkład wykładniczy z parametrem λ . Liczby jednostajne U_i generowane są za pomocą generatora LCG Lehmera, a następnie podstawiane do funkcji odwrotnej dystrybuanty.

6.2 Podstawy merytoryczne

Metoda odwracania dystrybuanty służy do generowania wartości zmiennej losowej o zadanym rozkładzie ciągłym. Jeżeli U ma rozkład jednostajny na przedziale $[0, 1)$, a F jest dystrybuantą wybranego rozkładu, to zmienna:

$$X = F^{-1}(U) \quad (58)$$

ma rozkład o dystrybuancie F .

Ogólny schemat metody:

$$U \sim U(0, 1), \quad X = F^{-1}(U). \quad (59)$$

W aktualnej implementacji zastosowano rozkład wykładniczy z parametrem λ . Jego dystrybuenta ma postać:

$$F(x) = \begin{cases} 0, & x < 0, \\ 1 - e^{-\lambda x}, & x \geq 0. \end{cases} \quad (60)$$

Aby wyznaczyć funkcję odwrotną:

$$u = 1 - e^{-\lambda x}, \quad (61)$$

$$e^{-\lambda x} = 1 - u, \quad (62)$$

$$-\lambda x = \ln(1 - u), \quad (63)$$

$$x = -\frac{\ln(1 - u)}{\lambda}. \quad (64)$$

Stąd:

$$X = -\frac{\ln(1 - U)}{\lambda}. \quad (65)$$

6.3 Algorytm

1. Wygenerować wartość U algorytmem LCG Lehmera.
2. Podstawić U do wzoru odwrotnej dystrybucyjności.
3. Otrzymać wartość X o rozkładzie wykładniczym.
4. Powtórzyć procedurę dla zadanej liczby próbek.

6.4 Fragment implementacji

```
uniform_value = min(max(generator.next_float(), 0.0), 0.9999999999999999)
value = -log(1 - uniform_value) / safe_lambda
```

6.5 Wyniki końcowe

Aplikacja prezentuje kolejne wartości U_i z generatora LCG, wygenerowane wartości $X_i = F^{-1}(U_i)$, średnią z próby, wariancję z próby oraz wartości teoretyczne:

$$E(X) = \frac{1}{\lambda}, \quad \text{Var}(X) = \frac{1}{\lambda^2}. \quad (66)$$

Jeżeli Przykład 1 z materiałów wykładowych definiuje inną dystrybucję niż rozkład wykładniczy, należy podmienić wzór funkcji odwrotnej w części implementacyjnej.

7 Podsumowanie

W projekcie opracowano trzy zadania z list laboratoryjnych. Każde zadanie zawiera podstawy teoretyczne, implementację algorytmu oraz prezentację wyników. Wspólnym elementem zadań symulacyjnych jest generator LCG Lehmera, który dostarcza liczby pseudolosowe z przedziału $[0, 1)$. Projekt spełnia wymaganie połączenia części merytorycznej z implementacją i wynikami końcowymi.